

Performance Analysis of API on On-Prem, EC2, and AWS Lambda

I. SYSTEM ARCHITECTURE

At the application layer, a Python API is developed using the Flask framework. This API is deployed on three platforms, making it accessible to multiple clients, including web applications, mobile apps, and testing pipelines. Once the deployed Python API was made accessible over the internet, JMeter was used to generate load and send requests to the API. Key performance indicators (KPIs), such as response time, latency, connection time, and CPU utilization, were measured and analyzed. These metrics were used to evaluate and compare the system's performance and scalability with a load of up to 50 TPS.

A. Python API Development

The application layer is built as a RESTful API using Python's Flask framework. This API has a dual-purpose design: a functioning CRUD service for structured data and a performance simulation environment that replicates realistic system characteristics.

This API has two main functionalities:

- 1) Basic CRUD operations on an in-memory dataset.
 - 2) Performance testing endpoints are designed to simulate different server behaviors.
- Health Check Endpoint
 - "/" → Returns a simple JSON response indicating that the API is active. This is primarily used to monitor the API's availability.
 - CRUD Endpoints

The API simulates a lightweight in-memory database (items =) and provides the standard Create, Read, Update, Delete (CRUD) functionality:

 - POST /items/ → Creates a new item, ensuring unique names.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - PUT /items/"name" → Updates an existing item with new data.
 - DELETE /items/"name" → Deletes an item by name.

This design offers a simple yet extensible data interaction model, suitable for experimentation and integration with downstream services.

B. Python API Deployment on On-Prem server

The steps mentioned below describe the end-to-end deployment workflow for hosting a Python API on an On-Prem server.

- On-Prem server with the configuration mentioned in Table 1 was used in this study.

OnPrem Server Configuration	
Processor	Intel(R) Core(TM) i3-2365M CPU @1.4GHz, 1400 Mhz
Installed RAM	4.00 GB
System Type	64-bit operating system, x64-based processor
Windows Specifications	
Windows Edition	Windows 10
Version	10.0.17134 Build 17134

TABLE I
ONPREM SERVER AND WINDOWS SPECIFICATIONS

- Installed Python 3.14 from the official Python website.

- Configured environment variables by adding the Python installation directory and the Scripts folder to the system PATH. PIP is a package installer for python.
- Allowed Python through the Windows Firewall to enable external access.
- Opened the required port on the server using command-line tools to allow the API to be reachable from outside the server. Below is the command that was used to allow incoming traffic on a specific TCP port (8000) through the Windows Firewall.

```
netsh advfirewall firewall show rule
name=all | findstr 8000
```

```
netsh advfirewall firewall
add rule name="Python_App"
dir=in action=allow protocol=TCP
localport=8000
```

- Deployed the API from the Command Prompt using the appropriate run command.
- The API is now successfully accessible over the internet.

C. Python API Deployment on AWS EC2 server

The steps mentioned below describe the end-to-end deployment workflow for hosting a Python API on an AWS Cloud server. Although most of the steps for deploying an API on a cloud server are similar to those on an On-Prem server, there is an additional step that needs to be completed: configuring inbound security rules to allow traffic on the port used by the API.

- AWS EC2 cloud server with the configuration mentioned in Table 2 was used in this study.

Cloud Server Configuration	
Processor	Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.5GHz 2.5 GHz
Installed RAM	2.00 GB
System Type	64-bit operating system, x64-based processor
Windows Specifications	
Windows Edition	Windows Server 2019 Datacenter
Version	10.0.17763 build 17763

TABLE II
AWS SERVER AND WINDOWS SPECIFICATIONS

- Installed Python 3.14 from the official Python website.
- Configured environment variables by adding the Python installation directory and the Scripts folder to the system PATH. PIP is a package installer for python.
- Allowed Python through the Windows Firewall to enable external access.
- Opened the required port on the server using command-line tools to allow the API to be reachable from outside the server. Below is the command that was used to allow incoming traffic on a specific TCP port (8000) through the Windows Firewall.

```
netsh advfirewall firewall show rule
name=all | findstr 8000
```

```
netsh advfirewall firewall
add rule name="Python_App"
dir=in action=allow protocol=TCP
localport=8000
```

- Deployed the API from the Command Prompt using the appropriate run command.
- These rules need to be added to prevent the AWS firewall from dropping all connections before they reach the Flask API.
 - On the EC2 instance → Security tab → Security groups.
 - Edit inbound rules → Add rule

- * Type: Custom TCP
- * Port range: 8000
- * Source: 0.0.0.0/0 (anywhere, for testing, if needed, we can assign a dedicated IP range for the traffic to reach the API)
- Save rules.
- The API is now successfully accessible over the internet.

D. Python API Deployment in Serverless Architecture

The Python API code used in the previous two tests was modified to make it compatible with AWS Lambda. The Lambda-deployed API was then exposed to the internet through API Gateway. Unlike the earlier tests, data was posted to a serverless DynamoDB table, since Lambda functions are stateless and can run across multiple instances. Each cold start produces a fresh execution environment with an empty in-memory dictionary. At a rate of around 50 TPS, Lambda can spawn multiple concurrent instances, each maintaining its own isolated memory for variables.

The steps mentioned below describe the end-to-end deployment workflow for hosting a Python API in a serverless architecture.

- Created a Lambda function to deploy the Python API, using the default VPC and security group.
- Created an API Gateway resource and integrated it with the Lambda function to handle all request methods.
- Enabled Lambda proxy integration to ensure full request and response pass through.
- Attached the CloudWatchLambdaInsightsExecutionRolePolicy and APIGatewayPushToCloudWatchLogs policies to allow Lambda and API Gateway to publish logs and metrics to CloudWatch.
- Created a DynamoDB table and updated the Lambda code to write incoming data to this table.
- Granted the Lambda function the AmazonDynamoDBFullAccess_v2 policy to allow read/write access to DynamoDB.

E. Executing Tests with JMeter

This JMeter script was designed to perform CRUD operations: Add, Get, Update, and Delete items using "HTTP Request" and "HTTP Header Manager" elements. Test data for the script was injected using the CSV Data Set Config element. Tests were executed on all three platforms to evaluate performance, latency, and scalability, with the load slowly ramping up to 50 TPS in 10 mins and sustained at 50 TPS for 5 mins.